have `welcome_email.text.erb` and `welcome_email.html.erb` in `app/views/user_mailer`, Action Mailer will automatically send a multipart email with the HTML and text versions setup as different parts. The order of the parts getting inserted is determined by the `:parts_order` inside of the `ActionMailer::Base.default` method.

2.9 Sending Emails with Dynamic Delivery Options

If you wish to override the default delivery options (e.g. SMTP credentials) while delivering emails, you can do this using `delivery_method_options` in the mailer action.

```
class UserMailer < ApplicationMailer
  def welcome_email(user, company)
    @user = user
    @url = user_url(@user)
    delivery_options = { user_name: company.smtp_user,
                         password: company.smtp_password,
                         address: company.smtp_host }
    mail(to: @user.email,
         subject: "Please see the Terms and Conditions attached",
         delivery_method_options: delivery_options)
  end
end
```

2.10 Sending Emails without Template Rendering

There may be cases in which you want to skip the template rendering step and supply the email body as a string. You can achieve this using the `:body` option. In such cases don't forget to add the `:content_type` option. Rails will default to `text/plain` otherwise.

```
class UserMailer < ApplicationMailer
  def welcome_email(user, email_body)
    mail(to: user.email,
         body: email_body,
         content_type: "text/html",
         subject: "Already rendered!")
  end
end
```

3 Receiving Emails

Receiving and parsing emails with Action Mailer can be a rather complex endeavor. Before your email reaches your Rails app, you would have had to configure your system to somehow forward emails to your app, which needs to be listening for that. So, to receive emails in your Rails app you'll need to:

- Implement a `receive` method in your mailer.
- Configure your email server to forward emails from the address(es) you would like your app to receive to `/path/to/app/bin/rails runner 'UserMailer.receive(STDIN.read)'`.

Once a method called `receive` is defined in any mailer, Action Mailer will parse the raw incoming email into an email object, decode it, instantiate a new mailer, and pass the email object to the mailer `receive` instance method. Here's an example:

```
class UserMailer < ApplicationMailer
  def receive(email)
    page = Page.find_by(address: email.to.first)
    page.emails.create(
      subject: email.subject,
      body: email.body
    )

    if email.has_attachments?
```